

Тестовое задание для AI / Data Science разработчика

Важно: ссылку на выполненное тестовое необходимо указать в резюме в блок «О себе».

Срок выполнения: 3 дня.

Формат сдачи: ссылка на публичный репозиторий (GitHub / GitLab).

Стек: Python 3.11+.

Бизнес-кейс: ИИ-помощник по складскому учёту и планированию закупок спа-оператора

Проект: Демонстрационный MVP встроенного ИИ-помощника, который на основе данных о движении товаров, текущих остатках, закупках, ценах, сроках поставки и сроках годности помогает планировать закупочную деятельность, контролировать состояние склада, прогнозировать потребность в товарах, рассчитывать рекомендуемый объём закупки, формировать бюджет закупок на месяц, квартал, полугодие и год и выявлять риски дефицита, излишков и списания.

Проблема (AS IS): информация о закупках, расходе, остатках, ценах и сроках годности находится в разных источниках и требует ручного анализа. Остатки и потребность анализируются вручную, прогноз закупки формируется на основе опыта сотрудника и простой оценки среднего расхода. В расчёте могут не учитываться открытые заказы, сезонность, рост цен, плановая загрузка и сроки годности. Риски дефицита и списания выявляются несистемно, а история расчёта и его обоснование могут не сохраняться.

Как процесс должен работать (TO BE)

- Пользователь открывает MVP и выбирает объект, период и категорию либо задаёт вопрос в чате.
- Система получает данные об остатках, расходе, закупках, ценах, поставках и сроках годности.
- Расчётный модуль прогнозирует потребность и проверяет точку заказа.
- AI-помощник формирует список рисков и рекомендуемый план закупок.
- Пользователь видит объём, стоимость, дату заказа, бюджет, объяснение расчёта и исходные показатели.
- Пользователь может изменить период, бюджетный лимит или другие параметры и пересчитать результат.
- Автоматическое размещение заказа в MVP не выполняется.

Отдельное требование заказчика: агент не должен самостоятельно придумывать остатки, цены или объёмы. Все числовые расчёты выполняются на основе структурированных данных и отображаются с указанием исходных параметров.

Ключевые сценарии использования (User Stories)

- Как управляющий, я хочу рассчитать объём и стоимость закупки конкретного товара на выбранный период, чтобы сформировать заказ и бюджет.
- Как управляющий, я хочу видеть товары с риском дефицита, чтобы расставить приоритеты и не допустить остановки процедур.
- Как управляющий, я хочу понимать причины изменения расчёта: рост спроса, увеличение цен, изменение расхода или страхового запаса, — а не получать голое число.
- Как пользователь, я хочу задавать вопросы обычным русским языком и получать ответ с расчётом и объяснением, уточняя период, филиал или бюджетный лимит.
- Как руководитель, я хочу контролировать долю ответов «требуется уточнение», чтобы понимать эффективность работы агента.

Задачи

Необходимо реализовать модуль интеллектуальной обработки складских данных, состоящий из трёх функций.

Для тестирования кода создайте в корне вашего репозитория файл dataset.json, скопировав в него данные из Приложения 1, и файл catalog.json — из Приложения 2.

Часть 1. Нормализация записей движения товара

- Реализовать функцию **normalize_movement(text: str) -> dict**, которая возвращает словарь с ключами: date (ISO-строка), sku, location, operation, qty (число в базовой единице позиции), unit, batch, doc_no.
- Поля, которые не удалось найти, должны возвращать None.
- Даты, количества и единицы встречаются в разных форматах — нужно покрыть основные (05.03.2026 / 1 марта 2026 г. / 03/06/26 / 2026-03-08 и 450 мл / 3,5 кг / 2 канистры по 5 л / 4 уп. по 50 пар). Количество приводится к базовой единице позиции из справочника (Приложение 2).
- Тип операции приходит словами и должен приводиться к одному из значений: receipt, consume, writeoff, return, correction. Корректировка может быть отрицательной.
- SKU встречается в разном написании (OIL-001, oil 001, «oil 001»), а иногда позиция названа только словами — сопоставление с справочником в этом случае на ваше усмотрение, решение обосновать в RESULTS.md.
- **Проверка:** прогоните normalize_movement на записях М1–М8 (Приложение 1) и отразите в RESULTS.md таблицу: что извлечено корректно, что пропущено.

Часть 2. Прогноз потребности и рекомендация закупки

- Реализовать функцию **forecast_demand(history: dict, sku: str, horizon_days: int, params: dict) -> dict**, которая возвращает: avg_daily_consumption, forecast_demand, current_stock, incoming_qty, safety_stock, reorder_point, recommended_qty, estimated_cost, stockout_date, confidence (от 0.0 до 1.0) и explanation — текстовое объяснение, из чего собрано число.
- Расчёт должен быть детерминированным и выполняться без обращения к LLM: языковая модель в этой части не участвует.
- Рекомендуемый объём округляется вверх до кратности упаковки и минимальной партии поставщика (Приложение 2), стоимость считается по цене из справочника.
- Метод прогнозирования — на ваш выбор (скользящее среднее, экспоненциальное сглаживание, регрессия с трендом и сезонностью). Выбор обосновать в RESULTS.md.
- Отдельно опишите, как обрабатываются пропуски в истории и выбросы: у одной позиции в данных пропущена неделя, у другой расход за последний месяц вырос более чем вдвое. Решение — считать это новым уровнем спроса или выбросом — принимаете вы, но оно должно быть обосновано.
- confidence должна снижаться при короткой истории и высокой волатильности расхода. Порог, ниже которого результат помечается как «требуется уточнение», подберите самостоятельно и обоснуйте в RESULTS.md.
- **Проверка:** прогоните функцию по трём позициям из Приложения 1 на горизонте 30 и 90 дней и покажите результат таблицей в RESULTS.md.

Часть 3. Разбор вопроса пользователя и объяснение ответа

- Реализовать функцию **answer_question(question: str, context: dict) -> tuple[dict, float, str]**, которая возвращает разобранные параметры запроса, уверенность от 0.0 до 1.0 и текст ответа с объяснением расчёта.
- Разобранные параметры включают интент и извлечённые сущности: sku, объект, период, бюджетный лимит. Возможные интенты: forecast_purchase, reorder_list, budget, deficit_risk, expiry_risk, price_dynamics, unknown.

- Если разрыв между первым и вторым интендом меньше выбранного порога либо в вопросе не хватает обязательных параметров — возвращать unknown и уточняющий вопрос, а не догадку.
- Числа в ответе берутся только из результата Части 2. Модель не имеет права придумывать остатки, цены и объёмы — это требование заказчика, и оно проверяется.
- **Подход:** рекомендуется использование LLM (через openai, gigachat, langchain или любой другой удобный клиент).
- **Обязательное условие:** предусмотреть fallback-механизм (например, правила и keyword matching), чтобы код можно было запустить локально без API-ключей.
- **Проверка:** прогоните функцию на 15 вопросах (Приложение 1) и покажите таблицей в RESULTS.md, что вернуло решение. Учитывайте EDGE-кейсы: вопросы без позиции, без периода и вне домена задачи.

Технические требования

- **Язык:** Python 3.11+.
- **Качество кода:** обязательны type hints, docstrings, соблюдение PEP8, отсутствие «магических чисел» — пороги и коэффициенты вынесены в конфигурацию.
- **Тестирование:** код должен быть покрыт базовыми тестами через pytest, включая разбор дат и единиц измерения и округление рекомендуемого объёма до упаковки.
- **Управление зависимостями:** requirements.txt / Pipfile / pyproject.toml.
- **Запуск:** прогон всего пайплайна на тестовых данных должен осуществляться 1–2 командами (например, простой CLI на argparse / click или python main.py).

Требования к репозиторию

- **README.md** — инструкция по запуску (установка зависимостей, запуск тестов, запуск пайплайна) и краткое описание архитектуры решения.
- **RESULTS.md** — результаты работы пайплайна: таблицы по всем трём частям, описание выбранных подходов, какие компромиссы сделаны, как обрабатываются пропуски и выбросы, обоснование порогов confidence и unknown.

Мотивация участия в проекте

- Почему тебе интересен этот проект?
- Как ты видишь свою роль в команде, которая создаёт продукт?
- Сколько времени в неделю ты готов(а) уделять проекту и в течение какого периода?

Приложение 1. Тестовые данные

Сохраните эти данные как dataset.json в вашем проекте.

Данные для Части 1 (записи движения товара M1–M8):

```
[
  {"id": "M1",
   "text": "05.03.2026 MS-01 приход OIL-001, 2 канистры по 5 л, НК-345"},
  {"id": "M2",
   "text": "1 марта 2026 MS-01 расход oil 001 — 450 мл, B-OIL-001-012"},
  {"id": "M3",
   "text": "03/06/26 Сочи списание SCRB-020 1,2 кг, истёк срок"},
  {"id": "M4",
   "text": "07.03.26 ms-02 WRAP 030 расход 3,5кг, парт.B-WRAP-030-004"},
  {"id": "M5",
   "text": "08.03.26 MS-01 приход OIL-001, 2 канистры по 5 л, НК-345"},
  {"id": "M6",
   "text": "09.03.26 MS-01 расход oil 001 — 450 мл, B-OIL-001-012"},
  {"id": "M7",
   "text": "10.03.26 MS-01 приход OIL-001, 2 канистры по 5 л, НК-345"},
  {"id": "M8",
   "text": "11.03.26 MS-01 расход oil 001 — 450 мл, B-OIL-001-012"}]
```

```
{
  "id": "M5",
  "text": "Возврат 12 марта 2026 г.: OIL-002, 2 л, брак упаковки"},
  "id": "M6",
  "text": "2026-03-08; MS-01; CONS-051; расход; 48 пар"},
  "id": "M7",
  "text": "Корректировка 15.03.2026, MS-02, CONS-052: -120 шт"},
  "id": "M8",
  "text": "Приход тапочек одноразовых, 4 уп. по 50 пар, Красная Поляна"}
]
```

Данные для Части 2 (недельный расход в базовых единицах за 12 недель, остатки и поставки в пути на 15.09.2026):

```
{
  "as_of": "2026-09-15",
  "weekly_consumption": {
    "OIL-001": [8.4, 9.1, 8.8, 9.6, 10.2, 9.4,
               11.0, 10.6, 12.3, 11.8, 12.9, 13.4],
    "SCRB-020": [3.1, 2.9, 3.4, 3.0, 3.3, 3.2,
                 3.5, 3.1, 6.8, 7.4, 7.1, 7.6],
    "WRAP-030": [6.2, 5.8, 6.5, 6.0, null, 6.3,
                 6.1, 6.4, 6.0, 6.6, 6.2, 6.9]
  },
  "current_stock": {"OIL-001": 50.4, "SCRB-020": 9.4,
                    "WRAP-030": 29.6},
  "incoming_qty": {"OIL-001": 20.0, "SCRB-020": 0.0,
                   "WRAP-030": 30.0}
}
```

Данные для Части 3 (15 вопросов пользователя):

```
[
  "Сколько масла закупить на три месяца и сколько это будет стоить?",
  "Что нужно заказать в ближайшие 14 дней?",
  "Какой бюджет закупок на квартал?",
  "Что закончится до следующей поставки в Сочи?",
  "Какие партии сгорят в этом месяце?",
  "Насколько подорожало ароматическое масло у поставщика?",
  "Сколько альгинатной маски осталось в Красной Поляне?",
  "Посчитай закупку скраба на полгода при лимите 200 тысяч",
  "Почему план закупок вырос по сравнению с прошлым кварталом?",
  "Сколько стоит закупить всё, что в риске дефицита?",
  "Сколько масла уйдёт за месяц, если загрузка вырастет на 20%?",
  "Заказать масло",
  "Сколько это будет стоить?",
]
```

```
"Когда привезут заказ от поставщика?",  
"Какая погода в Сочи на выходных?"  
]
```

Приложение 2. Справочник товаров

Сохраните эти данные как catalog.json. Цены указаны за базовую единицу позиции.

```
[  
  {  
    "sku": "OIL-001", "name": "Массажное масло базовое (миндаль)",  
    "unit": "л", "pack_size": 5, "min_order_qty": 10,  
    "safety_stock_days": 14, "lead_time_days": 7, "price": 1259.05  
  },  
  {  
    "sku": "OIL-002", "name": "Массажное масло ароматическое (лаванда)",  
    "unit": "л", "pack_size": 1, "min_order_qty": 5,  
    "safety_stock_days": 14, "lead_time_days": 7, "price": 3223.40  
  },  
  {  
    "sku": "SCRB-020", "name": "Скраб для тела кофейный",  
    "unit": "кг", "pack_size": 1, "min_order_qty": 5,  
    "safety_stock_days": 14, "lead_time_days": 10, "price": 1789.50  
  },  
  {  
    "sku": "WRAP-030", "name": "Альгинатная маска для обёртывания",  
    "unit": "кг", "pack_size": 1, "min_order_qty": 5,  
    "safety_stock_days": 14, "lead_time_days": 14, "price": 2437.00  
  },  
  {  
    "sku": "CONS-051", "name": "Тапочки одноразовые",  
    "unit": "пар", "pack_size": 50, "min_order_qty": 100,  
    "safety_stock_days": 10, "lead_time_days": 5, "price": 45.30  
  },  
  {  
    "sku": "CONS-052", "name": "Шапочки одноразовые",  
    "unit": "шт", "pack_size": 100, "min_order_qty": 200,  
    "safety_stock_days": 10, "lead_time_days": 5, "price": 9.70  
  }  
]
```